SA UVMF Guide

Annotated, source-complete walkthrough of the Serial Adder UVMF 2023.4 verification environment

Project baseline: SA_UVMF_2023_4_Custom_Integrated.zip | RTL: VHDL | Verification: SystemVerilog/UVMF 2023.4 | Questa mixed-language target

How to use this guide

Read Chapters 1–6 to follow stimulus and observations. Chapters 7–11 explain custom integration, configuration, coverage, and the build workflow. Appendix A contains the complete source of the key conversion files and the original source is retained in the project ZIP. Code reproduces the supplied custom-integrated archive. Static review only: successful generator execution, compile and simulation are not claimed.

Contents

1. Understanding the SA DUT and conversion boundaries

2. UVM versus UVMF architecture

3. Interface YAML and transaction fields

4. Sequence, agent, driver and BFM

5. Serial protocol timing and sampling

6. Input/output monitors and prediction

7. Custom predictor and scoreboard

8. Functional coverage and stimulus

9. Environment configuration and TLM wiring

10. Bench, DUT binding and test startup

11. Regeneration, Questa compilation and debugging

12. Verification checklist and open issues

Appendix A. Source listings (complete)

Appendix B. File map and reading order

1. Understanding the SA DUT and conversion boundaries

The DUT receives two 2-bit operands serialized most-significant bit first on one-bit ina/inb input pins. en_i identifies the first operand bit; result_o and en_o carry the 3-bit serial sum and output qualification. It has no reset input. The testbench reset used by generated UVMF BFMs is not a DUT port.

Port	Function	Verification responsibility
clk	Rising-edge DUT sample clock	Testbench clock and sampling discipline
ina, inb	Single-bit serialized operand inputs	Input driver BFM and monitor BFM
en_i	First operand bit request strobe	Active input driver
result_o	One-bit serial sum	Output monitor
en_o	Result qualification	Output monitor start detection

2. UVM versus UVMF architecture

Original UVM: sequence → sequencer → class driver → virtual interface → VHDL. UVMF: sequence → UVMF input agent/driver proxy → HDL driver BFM → DUT; HDL monitor BFM → HVL monitor → monitored_ap. Predictor/scoreboard/coverage remain custom UVM components integrated into the generated UVMF environment.

Stimulus: sa_in_random_sequence → sa_in_agent → sa_in_driver_bfm → VHDL sa. Observation: sa_in_monitor_bfm → sa_in_agent.monitored_ap → predictor + coverage; sa_out_monitor_bfm → sa_out_agent.monitored_ap → scoreboard actual FIFO. Predictor feeds expected FIFO.

3. Interface YAML and transaction fields

The four YAML specifications describe input and output interfaces, agent instances, and bench structure. YAML is source-of-generation; generated SystemVerilog is derived material. Generator output should not be confused with custom testbench wiring.

YAML	Purpose
sa_in_interface.yaml	2-bit input transaction and serial interface pins
sa_out_interface.yaml	3-bit result transaction and output signals
sa_environment.yaml	Input/output agents and environment
sa_bench.yaml	Bench-level structure, intended generation entry point

sa_in_interface.yaml

```
uvmf:
  interfaces:
    sa_in:
      clock: clk
      reset: rst_n
      reset_assertion_level: 'False'
      veloce_ready: 'False'
      ports:
        - {name: en_i, dir: output, width: '1', reset_value: "1'b0"}
        - {name: ina, dir: output, width: '1', reset_value: "1'b0"}
        - {name: inb, dir: output, width: '1', reset_value: "1'b0"}
      transaction_vars:
        - {name: ina, type: 'bit [1:0]', isrand: 'True', iscompare: 'True'}
        - {name: inb, type: 'bit [1:0]', isrand: 'True', iscompare: 'True'}
      config_vars: []
      config_constraints: []
      transaction_constraints: []
```

sa_out_interface.yaml

```
uvmf:
  interfaces:
    sa_out:
      clock: clk
      reset: rst_n
      reset_assertion_level: 'False'
      veloce_ready: 'False'
      ports:
        - {name: en_o, dir: input, width: '1', reset_value: "1'b0"}
        - {name: result_o, dir: input, width: '1', reset_value: "1'b0"}
      transaction_vars:
        - {name: result, type: 'bit [2:0]', isrand: 'False', iscompare: 'True'}
      config_vars: []
      config_constraints: []
      transaction_constraints: []
```

4. Sequence, agent, driver and BFM

A UVMF transaction is an HVL sequence item. The UVMF driver translates its fields into a BFM task call. The HDL BFM owns cycle-accurate pin driving. An input agent is active; an output agent acts as passive observation for this DUT.

Original SA UVM explicitly exercised convert2string, do_copy, do_compare, do_print, do_record, do_pack and do_unpack. The UVMF conversion does NOT independently demonstrate all seven overrides or their invocation; generated sa_in_transaction/sa_out_transaction are different transaction classes. Implement these hooks and invoke their public APIs if method parity is a requirement.

5. Serial protocol timing and sampling

The first input bit is the MSB, presented with en_i asserted. Second bit is LSB. VHDL consumes signals on rising edges. The input driver advances on opposite edges as implemented in the integrated BFM; the output monitor reconstructs three result bits beginning at en_o. Confirm timing in waveform prior to calling the protocol correct.

Relative phase	Input driving	Input observation	Output observation
First operand bit	en_i=1, ina[1],inb[1]	Capture MSBs	—
Second operand bit	en_i=0, ina[0],inb[0]	Capture LSBs / publish	—
Result qualification	Idle inputs	—	Detect en_o, capture first bit
Following result cycles	Idle inputs	—	Capture second and third bit / publish

6. Input/output monitors and prediction

The HDL monitor BFM reconstructs the parallel transaction and calls its UVMF proxy. The HVL monitor publishes transactions over monitored_ap. The input monitor branch feeds both the prediction stream and coverage. The output branch supplies observed serial sums. Distinct input/output transaction types are converted by the custom predictor and scoreboard as coded.

7. Custom predictor and scoreboard

The predictor receives sa_in_transaction, calculates an expected 3-bit unsigned sum, and publishes sa_out_transaction. The scoreboard consumes predicted and observed output streams via two analysis FIFOs, compares them in order, and maintains pass/fail counters. This FIFO-order assumption is suitable only when the DUT preserves transaction order.

sa_sa_predictor.svh

```
// Serial-adder reference model; called from monitor TLM analysis path.
`uvm_analysis_imp_decl(_sa_input)
class sa_sa_predictor extends uvm_component;
  `uvm_component_utils(sa_sa_predictor)
  uvm_analysis_imp_sa_input #(sa_in_transaction, sa_sa_predictor) input_export;
  uvm_analysis_port #(sa_out_transaction) expected_ap;
  function new(string name, uvm_component parent);
    super.new(name,parent);
    input_export = new("input_export",this);
    expected_ap = new("expected_ap",this);
  endfunction
  function void write_sa_input(sa_in_transaction t);
    sa_out_transaction e;
    e = sa_out_transaction::type_id::create("expected");
    e.result = {1'b0,t.ina} + {1'b0,t.inb};
    expected_ap.write(e);
  endfunction
endclass
```

sa_sa_scoreboard.svh

```
class sa_sa_scoreboard extends uvm_component;
 `uvm_component_utils(sa_sa_scoreboard)
 uvm_tlm_analysis_fifo #(sa_out_transaction) expected_fifo;
 uvm_tlm_analysis_fifo #(sa_out_transaction) actual_fifo;
 int unsigned pass_count, fail_count;
 function new(string name,uvm_component parent); super.new(name,parent); endfunction
 function void build_phase(uvm_phase phase);
   super.build_phase(phase);
   expected_fifo = new("expected_fifo",this);
   actual_fifo = new("actual_fifo",this);
 endfunction
 task run_phase(uvm_phase phase);
   sa_out_transaction exp, act;
   forever begin
     expected_fifo.get(exp);
     actual_fifo.get(act);
     if (exp.result === act.result) begin
       pass_count++;
       `uvm_info("SA_MATCH", $sformatf("sum=%0d",act.result), UVM_LOW)
     end else begin
       fail_count++;
       `uvm_error("SA_MISMATCH", $sformatf("expected=%0d actual=%0d",exp.result,act.result))
     end
```

```
    end
  endtask
  function void check_phase(uvm_phase phase);
    super.check_phase(phase);
    if (pass_count+fail_count == 0) `uvm_error("SA_NO_TEST","No transactions compared")
    if (expected_fifo.used() || actual_fifo.used()) `uvm_error("SA_REMAINDER","Unmatched transactions")
  endfunction
  function void report_phase(uvm_phase phase);
    `uvm_info("SA_RESULTS",$sformatf("pass=%0d fail=%0d",pass_count,fail_count),UVM_NONE)
  endfunction
endclass
```

8. Functional coverage and stimulus

2-bit A × 2-bit B yields 4 × 4 = 16 possible operand combinations. The custom coverage subscriber samples those combinations; the input sequence supplies exhaustive combinations followed by 15 random trials. Coverage is not a substitute for scoreboard checking.

sa_sa_coverage.svh

```
class sa_sa_coverage extends uvm_subscriber #(sa_in_transaction);
  `uvm_component_utils(sa_sa_coverage)
  bit [1:0] last_a, last_b;
  covergroup operands_cg;
    a_cp: coverpoint last_a { bins vals[] = {[0:3]}; }
    b_cp: coverpoint last_b { bins vals[] = {[0:3]}; }
    all_pairs: cross a_cp, b_cp;
  endgroup
  function new(string name, uvm_component parent);
    super.new(name,parent);
    operands_cg = new();
  endfunction
  function void write(sa_in_transaction t);
    last_a=t.ina;
    last_b=t.inb;
    operands_cg.sample();
  endfunction
endclass
```

sa_in_random_sequence.svh

```
//------------------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//------------------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//------------------------------------------------------------------------------
//------------------------------------------------------------------------------
//
// DESCRIPTION:
// This sequences randomizes the sa_in transaction and sends it
// to the UVM driver.
//
// This sequence constructs and randomizes a sa_in_transaction.
//
//------------------------------------------------------------------------------
//------------------------------------------------------------------------------
//
class sa_in_random_sequence
  extends sa_in_sequence_base ;

  `uvm_object_utils( sa_in_random_sequence )

  // pragma uvmf custom class_item_additional begin
  // pragma uvmf custom class_item_additional end

  //****************************************************************
  function new(string name = "");
    super.new(name);
  endfunction: new

  // ****************************************************************************
  // TASK : body()
  // This task is automatically executed when this sequence is started using the
  // start(sequencerHandle) task.
  //
  task body();
    // Exhaustively test 16 operand combinations (0..3 x 0..3)
    // followed by 15 randomized operand pairs.
    for (int a=0; a<4; a++) begin
      for (int b=0; b<4; b++) begin
        req = sa_in_transaction::type_id::create("exhaustive_req");
        start_item(req);
```

```
        req.ina = 2'(a);
        req.inb = 2'(b);
        finish_item(req);
      end
    end
    repeat (15) begin
      req = sa_in_transaction::type_id::create("random_req");
      start_item(req);
      if (!req.randomize()) `uvm_fatal("SA_RAND", "Cannot randomize")
      finish_item(req);
    end
  endtask

endclass: sa_in_random_sequence

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

9. Environment configuration and TLM wiring

The generated environment creates input and output agents and a virtual sequencer. Custom class items add sa_predictor, sa_scoreboard and sa_coverage. build_phase constructs these components and connect_phase wires analysis ports and FIFO exports. Recheck monitored_ap field naming against the installed UVMF 2023.4 base classes.

sa_environment.svh

```
//----------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//----------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//----------------------------------------------------------------------
//----------------------------------------------------------------------
//
// DESCRIPTION: This environment contains all agents, predictors and
// scoreboards required for the block level design.
//
//----------------------------------------------------------------------
//----------------------------------------------------------------------
//

class sa_environment  extends uvmf_environment_base #(
    .CONFIG_T( sa_env_configuration
  ));
  `uvm_component_utils( sa_environment )







  typedef sa_in_agent  sa_in_agent_t;
  sa_in_agent_t sa_in_agent;

  typedef sa_out_agent  sa_out_agent_t;
  sa_out_agent_t sa_out_agent;









  typedef uvmf_virtual_sequencer_base #(.CONFIG_T(sa_env_configuration)) sa_vsqr_t;
  sa_vsqr_t vsqr;

  // pragma uvmf custom class_item_additional begin
  sa_sa_predictor sa_predictor;
  sa_sa_scoreboard sa_scoreboard;
  sa_sa_coverage sa_coverage;
  // pragma uvmf custom class_item_additional end

// ****************************************************************************
// FUNCTION : new()
// This function is the standard SystemVerilog constructor.
```

```
//
  function new( string name = "", uvm_component parent = null );
    super.new( name, parent );
  endfunction

// ****************************************************************************
// FUNCTION: build_phase()
// This function builds all components within this environment.
//
  virtual function void build_phase(uvm_phase phase);
// pragma uvmf custom build_phase_pre_super begin
// pragma uvmf custom build_phase_pre_super end
    super.build_phase(phase);
    sa_in_agent = sa_in_agent_t::type_id::create("sa_in_agent",this);
    sa_in_agent.set_config(configuration.sa_in_agent_config);
    sa_out_agent = sa_out_agent_t::type_id::create("sa_out_agent",this);
    sa_out_agent.set_config(configuration.sa_out_agent_config);

    vsqr = sa_vsqr_t::type_id::create("vsqr", this);
    vsqr.set_config(configuration);
    configuration.set_vsqr(vsqr);

    // pragma uvmf custom build_phase begin
    sa_predictor = sa_sa_predictor::type_id::create("sa_predictor",this);
    sa_scoreboard = sa_sa_scoreboard::type_id::create("sa_scoreboard",this);
    sa_coverage = sa_sa_coverage::type_id::create("sa_coverage",this);
    // pragma uvmf custom build_phase end
  endfunction

// ****************************************************************************
// FUNCTION: connect_phase()
// This function makes all connections within this environment.  Connections
// typically inclue agent to predictor, predictor to scoreboard and scoreboard
// to agent.
//
  virtual function void connect_phase(uvm_phase phase);
// pragma uvmf custom connect_phase_pre_super begin
// pragma uvmf custom connect_phase_pre_super end
    super.connect_phase(phase);
    // Custom UVMF TLM integration: monitor -> predictor -> expected FIFO;
    //                              output monitor -> actual FIFO.
    sa_in_agent.monitored_ap.connect(sa_predictor.input_export);
    sa_in_agent.monitored_ap.connect(sa_coverage.analysis_export);
    sa_predictor.expected_ap.connect(sa_scoreboard.expected_fifo.analysis_export);
    sa_out_agent.monitored_ap.connect(sa_scoreboard.actual_fifo.analysis_export);
    // pragma uvmf custom reg_model_connect_phase begin
    // pragma uvmf custom reg_model_connect_phase end
  endfunction

// ****************************************************************************
// FUNCTION: end_of_simulation_phase()
// This function is executed just prior to executing run_phase.  This function
// was added to the environment to sample environment configuration settings
// just before the simulation exits time 0.  The configuration structure is
// randomized in the build phase before the environment structure is constructed.
// Configuration variables can be customized after randomization in the build_phase
// of the extended test.
// If a sequence modifies values in the configuration structure then the sequence is
// responsible for sampling the covergroup in the configuration if required.
//
  virtual function void start_of_simulation_phase(uvm_phase phase);
     configuration.sa_configuration_cg.sample();
  endfunction

endclass

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

sa_env_configuration.svh

```
//------------------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//------------------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//------------------------------------------------------------------------------
//------------------------------------------------------------------------------
//
// DESCRIPTION: THis is the configuration for the sa environment.
//  it contains configuration classes for each agent.  It also contains
//  environment level configuration variables.
//
//------------------------------------------------------------------------------
//------------------------------------------------------------------------------
//
```

```
class sa_env_configuration
extends uvmf_environment_configuration_base;

  `uvm_object_utils( sa_env_configuration )


//Constraints for the configuration variables:


  covergroup sa_configuration_cg;
    // pragma uvmf custom covergroup begin
    option.auto_bin_max=1024;
    // pragma uvmf custom covergroup end
  endgroup


    typedef sa_in_configuration sa_in_agent_config_t;
    rand sa_in_agent_config_t sa_in_agent_config;

    typedef sa_out_configuration sa_out_agent_config_t;
    rand sa_out_agent_config_t sa_out_agent_config;




  typedef uvmf_virtual_sequencer_base #(.CONFIG_T(sa_env_configuration)) sa_vsqr_t;
  sa_vsqr_t vsqr;

  // pragma uvmf custom class_item_additional begin
  // pragma uvmf custom class_item_additional end

// ****************************************************************************
// FUNCTION : new()
// This function is the standard SystemVerilog constructor.
// This function constructs the configuration object for each agent in the environment.
//
  function new( string name = "" );
    super.new( name );



    sa_in_agent_config = sa_in_agent_config_t::type_id::create("sa_in_agent_config");
    sa_out_agent_config = sa_out_agent_config_t::type_id::create("sa_out_agent_config");



    sa_configuration_cg=new;
    `uvm_warning("COVERAGE_MODEL_REVIEW", "A covergroup has been constructed which may need review because of either generation or re-
generation with merging.  Please note that configuration variables added as a result of re-generation and merging are not automatically added
to the covergroup.  Remove this warning after the covergroup has been reviewed.")

  // pragma uvmf custom new begin
  // pragma uvmf custom new end
  endfunction

// ****************************************************************************
// FUNCTION : set_vsqr()
// This function is used to assign the vsqr handle.
  virtual function void set_vsqr( sa_vsqr_t vsqr);
     this.vsqr = vsqr;
  endfunction : set_vsqr

// ****************************************************************************
// FUNCTION: post_randomize()
// This function is automatically called after the randomize() function
// is executed.
//
  function void post_randomize();
    super.post_randomize();
    // pragma uvmf custom post_randomize begin
    // pragma uvmf custom post_randomize end
  endfunction

// ****************************************************************************
// FUNCTION: convert2string()
// This function converts all variables in this class to a single string for
// logfile reporting. This function concatenates the convert2string result for
// each agent configuration in this configuration class.
//
  virtual function string convert2string();
    // pragma uvmf custom convert2string begin
    return {
```

```
      "\n", sa_in_agent_config.convert2string,
      "\n", sa_out_agent_config.convert2string


        };
    // pragma uvmf custom convert2string end
  endfunction
// ****************************************************************************
// FUNCTION: initialize();
// This function configures each interface agents configuration class.  The
// sim level determines the active/passive state of the agent.  The environment_path
// identifies the hierarchy down to and including the instantiation name of the
// environment for this configuration class.  Each instance of the environment
// has its own configuration class.  The string interface names are used by
// the agent configurations to identify the virtual interface handle to pull from
// the uvm_config_db.
//
  function void initialize(uvmf_sim_level_t sim_level,
                                    string environment_path,
                                    string interface_names[],
                                    uvm_reg_block register_model = null,
                                    uvmf_active_passive_t interface_activity[] = {}
                                   );

    super.initialize(sim_level, environment_path, interface_names, register_model, interface_activity);



  // Interface initialization for local agents
     sa_in_agent_config.initialize( interface_activity[0], {environment_path,".sa_in_agent"}, interface_names[0]);
     sa_in_agent_config.initiator_responder = INITIATOR;
     // sa_in_agent_config.has_coverage = 1;
     sa_out_agent_config.initialize( interface_activity[1], {environment_path,".sa_out_agent"}, interface_names[1]);
     sa_out_agent_config.initiator_responder = RESPONDER;
     // sa_out_agent_config.has_coverage = 1;






  // pragma uvmf custom initialize begin
  // pragma uvmf custom initialize end

  endfunction

endclass

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

10. Bench, DUT binding and test startup

Outstanding: the provided custom-integrated archive does not contain a completed generated bench/DUT top for compile-ready mixed-language simulation. Instantiate DUT sa, connect its five input/output signals and clock through the generated UVMF interface wiring. Do not map generated rst_n to the VHDL DUT, which has no reset. Start the randomized input sequence from a UVMF test or virtual sequence, raise/drop a run-phase objection, and drain the final output transaction.

11. Regeneration, Questa compilation and debugging

Use the generator scripts included in UVMF/scripts. Re-running yaml2uvmf.py can overwrite generated sources: preserve custom regions and compare regenerated diffs before committing. The supplied project has not been compiled or simulated under Questa.

Example generation command (run with a correctly installed UVMF_HOME; exact options may vary by local setup):

```
python "%UVMF_HOME%\scripts\yaml2uvmf.py" sa_in_interface.yaml sa_out_interface.yaml sa_environment.yaml sa_bench.yaml
```

Questa debugging priorities: YAML parser/generator diagnostics; HDL/HVL package compilation order; BFM task signatures; signal binding and VHDL elaboration; input monitored_ap events; expected/actual FIFO traffic; en_o sampling edge; objection lifespan; scoreboard totals and coverage.

12. Verification checklist and open issues

- YAML schema checked with installed UVMF 2023.4 generator — NOT VERIFIED
- Bench and DUT top-level connected — INCOMPLETE
- VHDL + SystemVerilog compile and elaboration — NOT VERIFIED
- Input BFM produces MSB-first operands — IMPLEMENTED, NOT SIMULATED
- Input and output monitors publish correct transactions — IMPLEMENTED, NOT SIMULATED
- Predictor and expected FIFO receive items — STATIC INTEGRATION
- Scoreboard reports full expected transaction count and zero mismatches — NOT VERIFIED
- 16 operand cross bins covered — INTENDED, NOT SIMULATED
- All seven UVM transaction public-method exercises — NOT ESTABLISHED FOR UVMF
- No incomplete FIFOs at end of run — NOT VERIFIED

Appendix A. Complete source listings

Each listing below is copied in full from SA_UVMF_2023_4_Custom_Integrated.zip. Generated package/library infrastructure is retained in the ZIP; selected protocol and custom integration files are reproduced here for line-by-line study.

sa.vhd

Source path: UVMF/rtl/sa.vhd

```
-- VHDL-2008 translation of original sa.v.
-- No reset: initial signal/variable values reproduce the Verilog initial block.
-- Port result_o replaces Verilog port "out" because OUT is a VHDL keyword.
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity sa is
  port (
    clk      : in  std_logic;
    en_i     : in  std_logic;
    ina      : in  std_logic;
    inb      : in  std_logic;
    en_o     : out std_logic := '0';
    result_o : out std_logic := '0'
  );
end entity sa;

architecture rtl of sa is
begin
  serial_adder_p : process (clk)
    variable counter  : integer := 0;
    variable state    : integer range 0 to 2 := 0;
    variable temp_a   : unsigned(1 downto 0) := (others => '0');
    variable temp_b   : unsigned(1 downto 0) := (others => '0');
    variable temp_out : unsigned(2 downto 0) := (others => '0');
  begin
    if rising_edge(clk) then
      -- Original RTL can restart operation whenever en_i is asserted.
      if en_i = '1' then
        state := 1;
      end if;

      case state is
        when 1 =>
          -- MSB first, two samples. Variables enforce sequential updates,
          -- matching blocking assignments in the source Verilog.
          temp_a := shift_left(temp_a, 1);
          temp_a(0) := ina;
          temp_b := shift_left(temp_b, 1);
          temp_b(0) := inb;
          counter := counter + 1;
          if counter = 2 then
            temp_out := resize(temp_a, 3) + resize(temp_b, 3);
            state := 2;
          end if;

        when 2 =>
          -- The source updates output ports with nonblocking assignments.
          result_o <= temp_out(2);
          temp_out := shift_left(temp_out, 1);
          counter := counter + 1;
          if counter = 3 then
            en_o <= '1';
```

```
          end if;
          if counter = 4 then
            en_o <= '0';
          end if;
          if counter = 6 then
            counter := 0;
            result_o <= '0';
            state := 0;
          end if;

        when others =>
          null;
      end case;
    end if;
  end process;
end architecture rtl;
```

sa_in_interface.yaml

Source path: UVMF/yaml/sa_in_interface.yaml

```
uvmf:
  interfaces:
    sa_in:
      clock: clk
      reset: rst_n
      reset_assertion_level: 'False'
      veloce_ready: 'False'
      ports:
        - {name: en_i, dir: output, width: '1', reset_value: "1'b0"}
        - {name: ina, dir: output, width: '1', reset_value: "1'b0"}
        - {name: inb, dir: output, width: '1', reset_value: "1'b0"}
      transaction_vars:
        - {name: ina, type: 'bit [1:0]', isrand: 'True', iscompare: 'True'}
        - {name: inb, type: 'bit [1:0]', isrand: 'True', iscompare: 'True'}
      config_vars: []
      config_constraints: []
      transaction_constraints: []
```

sa_out_interface.yaml

Source path: UVMF/yaml/sa_out_interface.yaml

```
uvmf:
  interfaces:
    sa_out:
      clock: clk
      reset: rst_n
      reset_assertion_level: 'False'
      veloce_ready: 'False'
      ports:
        - {name: en_o, dir: input, width: '1', reset_value: "1'b0"}
        - {name: result_o, dir: input, width: '1', reset_value: "1'b0"}
      transaction_vars:
        - {name: result, type: 'bit [2:0]', isrand: 'False', iscompare: 'True'}
      config_vars: []
      config_constraints: []
      transaction_constraints: []
```

sa_environment.yaml

Source path: UVMF/yaml/sa_environment.yaml

```
uvmf:
  environments:
    sa:
      agents:
        - {name: sa_in_agent, type: sa_in, initiator_responder: INITIATOR}
        - {name: sa_out_agent, type: sa_out, initiator_responder: RESPONDER}
      analysis_components: []
      scoreboards: []
      subenvs: []
```

sa_bench.yaml

Source path: UVMF/yaml/sa_bench.yaml

```
uvmf:
  benches:
    sa:
      top_env: sa
      clock_half_period: 5ns
      clock_phase_offset: 0ns
      reset_assertion_level: 'False'
      reset_duration: 50ns
      active_passive:
        - {bfm_name: sa_in_agent, value: ACTIVE}
        - {bfm_name: sa_out_agent, value: PASSIVE}
```

```
      interface_params: []
```

sa_in_transaction.svh

Source path: UVMF/verification_ip/interface_packages/sa_in_pkg/src/sa_in_transaction.svh

```
//----------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//----------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//----------------------------------------------------------------------
//----------------------------------------------------------------------
//
// DESCRIPTION: This class defines the variables required for an sa_in
//    transaction.  Class variables to be displayed in waveform transaction
//    viewing are added to the transaction viewing stream in the add_to_wave
//    function.
//
//----------------------------------------------------------------------
//----------------------------------------------------------------------
//
class sa_in_transaction  extends uvmf_transaction_base;

  `uvm_object_utils( sa_in_transaction )

  rand bit [1:0] ina ;
  rand bit [1:0] inb ;

  //Constraints for the transaction variables:

  // pragma uvmf custom class_item_additional begin
  // pragma uvmf custom class_item_additional end

  // ****************************************************************************
  // FUNCTION : new()
  // This function is the standard SystemVerilog constructor.
  //
  function new( string name = "" );
    super.new( name );
  endfunction

  // ****************************************************************************
  // FUNCTION: convert2string()
  // This function converts all variables in this class to a single string for
  // logfile reporting.
  //
  virtual function string convert2string();
    // pragma uvmf custom convert2string begin
    // UVMF_CHANGE_ME : Customize format if desired.
    return $sformatf("ina:0x%x inb:0x%x ",ina,inb);
    // pragma uvmf custom convert2string end
  endfunction

  //****************************************************************
  // FUNCTION: do_print()
  // This function is automatically called when the .print() function
  // is called on this class.
  //
  virtual function void do_print(uvm_printer printer);
    // pragma uvmf custom do_print begin
    // UVMF_CHANGE_ME : Current contents of do_print allows for the use of UVM 1.1d, 1.2 or P1800.2.
    // Update based on your own printing preference according to your preferred UVM version
    $display(convert2string());
    // pragma uvmf custom do_print end
  endfunction

  //****************************************************************
  // FUNCTION: do_compare()
  // This function is automatically called when the .compare() function
  // is called on this class.
  //
  virtual function bit do_compare (uvm_object rhs, uvm_comparer comparer);
    sa_in_transaction  RHS;
    if (!$cast(RHS,rhs)) return 0;
    // pragma uvmf custom do_compare begin
    // UVMF_CHANGE_ME : Eliminate comparison of variables not to be used for compare
    return (super.do_compare(rhs,comparer)
            &&(this.ina == RHS.ina)
            &&(this.inb == RHS.inb)
            );
    // pragma uvmf custom do_compare end
  endfunction

  //****************************************************************
  // FUNCTION: do_copy()
```

```
  // This function is automatically called when the .copy() function
  // is called on this class.
  //
  virtual function void do_copy (uvm_object rhs);
    sa_in_transaction  RHS;
    if(!$cast(RHS,rhs))begin
      `uvm_fatal("CAST","Transaction cast in do_copy() failed!")
    end
    // pragma uvmf custom do_copy begin
    super.do_copy(rhs);
    this.ina = RHS.ina;
    this.inb = RHS.inb;
    // pragma uvmf custom do_copy end
  endfunction

  // ****************************************************************************
  // FUNCTION: add_to_wave()
  // This function is used to display variables in this class in the waveform
  // viewer.  The start_time and end_time variables must be set before this
  // function is called.  If the start_time and end_time variables are not set
  // the transaction will be hidden at 0ns on the waveform display.
  //
  virtual function void add_to_wave(int transaction_viewing_stream_h);
    `ifdef QUESTA
    if (transaction_view_h == 0) begin
      transaction_view_h = $begin_transaction(transaction_viewing_stream_h,"sa_in_transaction",start_time);
    end
    super.add_to_wave(transaction_view_h);
    // pragma uvmf custom add_to_wave begin
    // UVMF_CHANGE_ME : Color can be applied to transaction entries based on content, example below
    // case()
    //   1 : $add_color(transaction_view_h,"red");
    //   default : $add_color(transaction_view_h,"grey");
    // endcase
    // UVMF_CHANGE_ME : Eliminate transaction variables not wanted in transaction viewing in the waveform viewer
    $add_attribute(transaction_view_h,ina,"ina");
    $add_attribute(transaction_view_h,inb,"inb");
    // pragma uvmf custom add_to_wave end
    $end_transaction(transaction_view_h,end_time);
    $free_transaction(transaction_view_h);
    `endif // QUESTA
  endfunction

endclass

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

sa_out_transaction.svh

Source path: UVMF/verification_ip/interface_packages/sa_out_pkg/src/sa_out_transaction.svh

```
//------------------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//------------------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//------------------------------------------------------------------------------
//------------------------------------------------------------------------------
//
// DESCRIPTION: This class defines the variables required for an sa_out
//    transaction.  Class variables to be displayed in waveform transaction
//    viewing are added to the transaction viewing stream in the add_to_wave
//    function.
//
//------------------------------------------------------------------------------
//------------------------------------------------------------------------------
//
class sa_out_transaction  extends uvmf_transaction_base;

  `uvm_object_utils( sa_out_transaction )

  bit [2:0] result ;

  //Constraints for the transaction variables:

  // pragma uvmf custom class_item_additional begin
  // pragma uvmf custom class_item_additional end

  // ****************************************************************************
  // FUNCTION : new()
  // This function is the standard SystemVerilog constructor.
  //
  function new( string name = "" );
    super.new( name );
```

```
  endfunction

  // ****************************************************************************
  // FUNCTION: convert2string()
  // This function converts all variables in this class to a single string for
  // logfile reporting.
  //
  virtual function string convert2string();
    // pragma uvmf custom convert2string begin
    // UVMF_CHANGE_ME : Customize format if desired.
    return $sformatf("result:0x%x ",result);
    // pragma uvmf custom convert2string end
  endfunction

  //****************************************************************
  // FUNCTION: do_print()
  // This function is automatically called when the .print() function
  // is called on this class.
  //
  virtual function void do_print(uvm_printer printer);
    // pragma uvmf custom do_print begin
    // UVMF_CHANGE_ME : Current contents of do_print allows for the use of UVM 1.1d, 1.2 or P1800.2.
    // Update based on your own printing preference according to your preferred UVM version
    $display(convert2string());
    // pragma uvmf custom do_print end
  endfunction

  //****************************************************************
  // FUNCTION: do_compare()
  // This function is automatically called when the .compare() function
  // is called on this class.
  //
  virtual function bit do_compare (uvm_object rhs, uvm_comparer comparer);
    sa_out_transaction  RHS;
    if (!$cast(RHS,rhs)) return 0;
    // pragma uvmf custom do_compare begin
    // UVMF_CHANGE_ME : Eliminate comparison of variables not to be used for compare
    return (super.do_compare(rhs,comparer)
            &&(this.result == RHS.result)
            );
    // pragma uvmf custom do_compare end
  endfunction

  //****************************************************************
  // FUNCTION: do_copy()
  // This function is automatically called when the .copy() function
  // is called on this class.
  //
  virtual function void do_copy (uvm_object rhs);
    sa_out_transaction  RHS;
    if(!$cast(RHS,rhs))begin
      `uvm_fatal("CAST","Transaction cast in do_copy() failed!")
    end
    // pragma uvmf custom do_copy begin
    super.do_copy(rhs);
    this.result = RHS.result;
    // pragma uvmf custom do_copy end
  endfunction

  // ****************************************************************************
  // FUNCTION: add_to_wave()
  // This function is used to display variables in this class in the waveform
  // viewer.  The start_time and end_time variables must be set before this
  // function is called.  If the start_time and end_time variables are not set
  // the transaction will be hidden at 0ns on the waveform display.
  //
  virtual function void add_to_wave(int transaction_viewing_stream_h);
    `ifdef QUESTA
    if (transaction_view_h == 0) begin
      transaction_view_h = $begin_transaction(transaction_viewing_stream_h,"sa_out_transaction",start_time);
    end
    super.add_to_wave(transaction_view_h);
    // pragma uvmf custom add_to_wave begin
    // UVMF_CHANGE_ME : Color can be applied to transaction entries based on content, example below
    // case()
    //   1 : $add_color(transaction_view_h,"red");
    //   default : $add_color(transaction_view_h,"grey");
    // endcase
    // UVMF_CHANGE_ME : Eliminate transaction variables not wanted in transaction viewing in the waveform viewer
    $add_attribute(transaction_view_h,result,"result");
    // pragma uvmf custom add_to_wave end
    $end_transaction(transaction_view_h,end_time);
    $free_transaction(transaction_view_h);
    `endif // QUESTA
  endfunction

endclass
```

```
// pragma uvmf custom external begin
// pragma uvmf custom external end
```

sa_in_driver_bfm.sv

Source path: UVMF/verification_ip/interface_packages/sa_in_pkg/src/sa_in_driver_bfm.sv

```
//------------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//------------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//------------------------------------------------------------------------
//------------------------------------------------------------------------
//
// DESCRIPTION:
//    This interface performs the sa_in signal driving.  It is
//     accessed by the uvm sa_in driver through a virtual interface
//     handle in the sa_in configuration.  It drives the singals passed
//     in through the port connection named bus of type sa_in_if.
//
//     Input signals from the sa_in_if are assigned to an internal input
//     signal with a _i suffix.  The _i signal should be used for sampling.
//
//     The input signal connections are as follows:
//       bus.signal -> signal_i
//
//     This bfm drives signals with a _o suffix.  These signals
//     are driven onto signals within sa_in_if based on INITIATOR/RESPONDER and/or
//     ARBITRATION/GRANT status.
//
//     The output signal connections are as follows:
//        signal_o -> bus.signal
//
//
//      Interface functions and tasks used by UVM components:
//
//             configure:
//                   This function gets configuration attributes from the
//                   UVM driver to set any required BFM configuration
//                   variables such as 'initiator_responder'.
//
//             initiate_and_get_response:
//                   This task is used to perform signaling activity for initiating
//                   a protocol transfer.  The task initiates the transfer, using
//                   input data from the initiator struct.  Then the task captures
//                   response data, placing the data into the response struct.
//                   The response struct is returned to the driver class.
//
//             respond_and_wait_for_next_transfer:
//                   This task is used to complete a current transfer as a responder
//                   and then wait for the initiator to start the next transfer.
//                   The task uses data in the responder struct to drive protocol
//                   signals to complete the transfer.  The task then waits for
//                   the next transfer.  Once the next transfer begins, data from
//                   the initiator is placed into the initiator struct and sent
//                   to the responder sequence for processing to determine
//                   what data to respond with.
//
//------------------------------------------------------------------------
//------------------------------------------------------------------------
//
import uvmf_base_pkg_hdl::*;
import sa_in_pkg_hdl::*;
import sa_in_pkg::*;

interface sa_in_driver_bfm
  (sa_in_if bus);

`ifndef XRTL
// This code is to aid in debugging parameter mismatches between the BFM and its corresponding agent.
// Enable this debug by setting UVM_VERBOSITY to UVM_DEBUG
// Setting UVM_VERBOSITY to UVM_DEBUG causes all BFM's and all agents to display their parameter settings.
// All of the messages from this feature have a UVM messaging id value of "CFG"
// The transcript or run.log can be parsed to ensure BFM parameter settings match its corresponding agents parameter settings.
import uvm_pkg::*;
`include "uvm_macros.svh"
initial begin : bfm_vs_agent_parameter_debug
  `uvm_info("CFG",
      $psprintf("The BFM at '%m' has the following parameters: ", ),
      UVM_DEBUG)
end
`endif

  // Config value to determine if this is an initiator or a responder
  uvmf_initiator_responder_t initiator_responder;
```

```
  // Custom configuration variables.
  // These are set using the configure function which is called during the UVM connect_phase

  tri clk_i;
  tri rst_n_i;

  // Signal list (all signals are capable of being inputs and outputs for the sake
  // of supporting both INITIATOR and RESPONDER mode operation. Expectation is that
  // directionality in the config file was from the point-of-view of the INITIATOR

  // INITIATOR mode input signals

  // INITIATOR mode output signals
  tri  en_i_i;
  reg  en_i_o = 1'b0;
  tri  ina_i;
  reg  ina_o = 1'b0;
  tri  inb_i;
  reg  inb_o = 1'b0;

  // Bi-directional signals



  assign clk_i = bus.clk;
  assign rst_n_i = bus.rst_n;

  // These are signals marked as 'input' by the config file, but the signals will be
  // driven by this BFM if put into RESPONDER mode (flipping all signal directions around)



  // These are signals marked as 'output' by the config file, but the outputs will
  // not be driven by this BFM unless placed in INITIATOR mode.
  assign bus.en_i = (initiator_responder == INITIATOR) ? en_i_o : 'bz;
  assign en_i_i = bus.en_i;
  assign bus.ina = (initiator_responder == INITIATOR) ? ina_o : 'bz;
  assign ina_i = bus.ina;
  assign bus.inb = (initiator_responder == INITIATOR) ? inb_o : 'bz;
  assign inb_i = bus.inb;

  // Proxy handle to UVM driver
  sa_in_pkg::sa_in_driver   proxy;



  // ****************************************************************************
// pragma uvmf custom reset_condition_and_response begin
  // Always block used to return signals to reset value upon assertion of reset
  always @( negedge rst_n_i )
     begin
       // RESPONDER mode output signals
       // INITIATOR mode output signals
       en_i_o <= 1'b0;
       ina_o <= 1'b0;
       inb_o <= 1'b0;
       // Bi-directional signals

     end
// pragma uvmf custom reset_condition_and_response end

  // pragma uvmf custom interface_item_additional begin
  // pragma uvmf custom interface_item_additional end

  //***************************************************************
  // The configure() function is used to pass agent configuration
  // variables to the driver BFM.  It is called by the driver within
  // the agent at the beginning of the simulation.  It may be called
  // during the simulation if agent configuration variables are updated
  // and the driver BFM needs to be aware of the new configuration
  // variables.
  //

  function void configure(sa_in_configuration

                          sa_in_configuration_arg
                          );
    initiator_responder = sa_in_configuration_arg.initiator_responder;
  // pragma uvmf custom configure begin
  // pragma uvmf custom configure end
  endfunction

// pragma uvmf custom initiate_and_get_response begin
  // Serial adder: drive MSB, then LSB, on falling clock edges.
  // Allow the six rising-edge DUT stages to finish before next request.
```

```
  task initiate_and_get_response(sa_in_transaction initiator_trans);
    @(negedge clk_i);
    en_i_o <= 1'b1;
    ina_o  <= initiator_trans.ina[1];
    inb_o  <= initiator_trans.inb[1];
    @(negedge clk_i);
    en_i_o <= 1'b0;
    ina_o  <= initiator_trans.ina[0];
    inb_o  <= initiator_trans.inb[0];
    @(negedge clk_i);
    ina_o <= 1'b0;
    inb_o <= 1'b0;
    repeat (5) @(negedge clk_i);
  endtask
// pragma uvmf custom initiate_and_get_response end

// pragma uvmf custom respond_and_wait_for_next_transfer begin
// ****************************************************************************
// The first_transfer variable is used to prevent completing a transfer in the
// first call to this task.  For the first call to this task, there is not
// current transfer to complete.
bit first_transfer=1;

// UVMF_CHANGE_ME
// This task is used by a responder.  The task first completes the current
// transfer in progress then waits for the initiator to start the next transfer.

  task respond_and_wait_for_next_transfer( sa_in_transaction

                                           responder_trans
                                           );
  // Variables within the responder_trans:
  //   bit [1:0] ina ;
  //   bit [1:0] inb ;
       // Reference code;
       //    How to wait for signal value
       //      while (control_signal == 1'b1) @(posedge clk_i);
       //
       //    How to assign a responder_trans member, named xyz, from a signal.
       //    All available responder input and inout signals listed.
       //    Responder input signals
       //      responder_trans.xyz = en_i_i;  //
       //      responder_trans.xyz = ina_i;  //
       //      responder_trans.xyz = inb_i;  //
       //    Responder inout signals
       //    How to assign a signal from a responder_trans member named xyz.
       //    All available responder output and inout signals listed.
       //    Notice the _o.  Those are storage variables that allow for procedural assignment.
       ///   Responder output signals
       //    Responder inout signals


  @(posedge clk_i);
  if (!first_transfer) begin
    // Perform transfer response here.
    // Reply using data recieved in the responder_trans.
    @(posedge clk_i);
    // Reply using data recieved in the transaction handle.
    @(posedge clk_i);
  end
    // Wait for next transfer then gather info from intiator about the transfer.
    // Place the data into the responder_trans handle.
    @(posedge clk_i);
    @(posedge clk_i);
    first_transfer = 0;
  endtask
// pragma uvmf custom respond_and_wait_for_next_transfer end


endinterface

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

sa_in_monitor_bfm.sv

Source path: UVMF/verification_ip/interface_packages/sa_in_pkg/src/sa_in_monitor_bfm.sv

```
//------------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//------------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//------------------------------------------------------------------------
//------------------------------------------------------------------------
```

```
//
// DESCRIPTION: This interface performs the sa_in signal monitoring.
//      It is accessed by the uvm sa_in monitor through a virtual
//      interface handle in the sa_in configuration.  It monitors the
//      signals passed in through the port connection named bus of
//      type sa_in_if.
//
//     Input signals from the sa_in_if are assigned to an internal input
//     signal with a _i suffix.  The _i signal should be used for sampling.
//
//     The input signal connections are as follows:
//       bus.signal -> signal_i
//
//      Interface functions and tasks used by UVM components:
//            monitor(inout TRANS_T txn);
//                  This task receives the transaction, txn, from the
//                  UVM monitor and then populates variables in txn
//                  from values observed on bus activity.  This task
//                  blocks until an operation on the sa_in bus is complete.
//
//------------------------------------------------------------------------
//------------------------------------------------------------------------
//
import uvmf_base_pkg_hdl::*;
import sa_in_pkg_hdl::*;
import sa_in_pkg::*;



interface sa_in_monitor_bfm
  ( sa_in_if  bus );

`ifndef XRTL
// This code is to aid in debugging parameter mismatches between the BFM and its corresponding agent.
// Enable this debug by setting UVM_VERBOSITY to UVM_DEBUG
// Setting UVM_VERBOSITY to UVM_DEBUG causes all BFM's and all agents to display their parameter settings.
// All of the messages from this feature have a UVM messaging id value of "CFG"
// The transcript or run.log can be parsed to ensure BFM parameter settings match its corresponding agents parameter settings.
import uvm_pkg::*;
`include "uvm_macros.svh"
initial begin : bfm_vs_agent_parameter_debug
  `uvm_info("CFG",
      $psprintf("The BFM at '%m' has the following parameters: ", ),
      UVM_DEBUG)
end
`endif



 sa_in_transaction
                     monitored_trans;


  // Config value to determine if this is an initiator or a responder
  uvmf_initiator_responder_t initiator_responder;
  // Custom configuration variables.
  // These are set using the configure function which is called during the UVM connect_phase

  tri clk_i;
  tri rst_n_i;
  tri  en_i_i;
  tri  ina_i;
  tri  inb_i;
  assign clk_i = bus.clk;
  assign rst_n_i = bus.rst_n;
  assign en_i_i = bus.en_i;
  assign ina_i = bus.ina;
  assign inb_i = bus.inb;

  // Proxy handle to UVM monitor
  sa_in_pkg::sa_in_monitor  proxy;

  // pragma uvmf custom interface_item_additional begin
  // pragma uvmf custom interface_item_additional end

  //****************************************************************
  task wait_for_reset();
    @(posedge clk_i) ;
    do_wait_for_reset();
  endtask

  // ****************************************************************************
  task do_wait_for_reset();
  // pragma uvmf custom reset_condition begin
    wait ( rst_n_i === 1 ) ;
    @(posedge clk_i) ;
  // pragma uvmf custom reset_condition end
```

```
  endtask

  //********************************************************************

  task wait_for_num_clocks(input int unsigned count);
    @(posedge clk_i);

    repeat (count-1) @(posedge clk_i);
  endtask

  //********************************************************************
  event go;
  function void start_monitoring();
    -> go;
  endfunction

  // ****************************************************************************
  initial begin
    @go;
    forever begin
      @(posedge clk_i);
      monitored_trans = new("monitored_trans");
      do_monitor( );



      proxy.notify_transaction( monitored_trans );

    end
  end

  //********************************************************************
  // The configure() function is used to pass agent configuration
  // variables to the monitor BFM.  It is called by the monitor within
  // the agent at the beginning of the simulation.  It may be called
  // during the simulation if agent configuration variables are updated
  // and the monitor BFM needs to be aware of the new configuration
  // variables.
  //
    function void configure(sa_in_configuration

                            sa_in_configuration_arg
                            );
    initiator_responder = sa_in_configuration_arg.initiator_responder;
  // pragma uvmf custom configure begin
  // pragma uvmf custom configure end
  endfunction



  // ****************************************************************************
  task do_monitor();
    //
    // Available struct members:
    //     //    monitored_trans.ina
    //     //    monitored_trans.inb
    //     //
    // Reference code;
    //    How to wait for signal value
    //      while (control_signal === 1'b1) @(posedge clk_i);
    //
    //    How to assign a transaction variable, named xyz, from a signal.
    //    All available input signals listed.
    //      monitored_trans.xyz = en_i_i;  //
    //      monitored_trans.xyz = ina_i;  //
    //      monitored_trans.xyz = inb_i;  //
    // pragma uvmf custom do_monitor begin
    // UVMF monitor task is entered just after a positive edge.
    // Capture first bit while enable is asserted, second on following edge.
    monitored_trans.start_time = $time;
    do @(negedge clk_i); while (en_i_i !== 1'b1);
    monitored_trans.ina[1] = ina_i;
    monitored_trans.inb[1] = inb_i;
    @(negedge clk_i);
    monitored_trans.ina[0] = ina_i;
    monitored_trans.inb[0] = inb_i;
    monitored_trans.end_time = $time;

    // pragma uvmf custom do_monitor end
  endtask



endinterface

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

sa_out_monitor_bfm.sv

Source path: UVMF/verification_ip/interface_packages/sa_out_pkg/src/sa_out_monitor_bfm.sv

```
//------------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//------------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//------------------------------------------------------------------------
//------------------------------------------------------------------------
//
// DESCRIPTION: This interface performs the sa_out signal monitoring.
//      It is accessed by the uvm sa_out monitor through a virtual
//      interface handle in the sa_out configuration.  It monitors the
//      signals passed in through the port connection named bus of
//      type sa_out_if.
//
//     Input signals from the sa_out_if are assigned to an internal input
//     signal with a _i suffix.  The _i signal should be used for sampling.
//
//     The input signal connections are as follows:
//       bus.signal -> signal_i
//
//      Interface functions and tasks used by UVM components:
//             monitor(inout TRANS_T txn);
//                   This task receives the transaction, txn, from the
//                   UVM monitor and then populates variables in txn
//                   from values observed on bus activity.  This task
//                   blocks until an operation on the sa_out bus is complete.
//
//------------------------------------------------------------------------
//------------------------------------------------------------------------
//
import uvmf_base_pkg_hdl::*;
import sa_out_pkg_hdl::*;
import sa_out_pkg::*;



interface sa_out_monitor_bfm
  ( sa_out_if  bus );

`ifndef XRTL
// This code is to aid in debugging parameter mismatches between the BFM and its corresponding agent.
// Enable this debug by setting UVM_VERBOSITY to UVM_DEBUG
// Setting UVM_VERBOSITY to UVM_DEBUG causes all BFM's and all agents to display their parameter settings.
// All of the messages from this feature have a UVM messaging id value of "CFG"
// The transcript or run.log can be parsed to ensure BFM parameter settings match its corresponding agents parameter settings.
import uvm_pkg::*;
`include "uvm_macros.svh"
initial begin : bfm_vs_agent_parameter_debug
  `uvm_info("CFG",
      $psprintf("The BFM at '%m' has the following parameters: ", ),
      UVM_DEBUG)
end
`endif



 sa_out_transaction
                     monitored_trans;



  // Config value to determine if this is an initiator or a responder
  uvmf_initiator_responder_t initiator_responder;
  // Custom configuration variables.
  // These are set using the configure function which is called during the UVM connect_phase


  tri clk_i;
  tri rst_n_i;
  tri  en_o_i;
  tri  result_o_i;
  assign clk_i = bus.clk;
  assign rst_n_i = bus.rst_n;
  assign en_o_i = bus.en_o;
  assign result_o_i = bus.result_o;

  // Proxy handle to UVM monitor
  sa_out_pkg::sa_out_monitor  proxy;

  // pragma uvmf custom interface_item_additional begin
  // pragma uvmf custom interface_item_additional end

  //***************************************************************
```

```
  task wait_for_reset();
    @(posedge clk_i) ;
    do_wait_for_reset();
  endtask

  // ****************************************************************************
  task do_wait_for_reset();
  // pragma uvmf custom reset_condition begin
    wait ( rst_n_i === 1 ) ;
    @(posedge clk_i) ;
  // pragma uvmf custom reset_condition end
  endtask

  //****************************************************************

  task wait_for_num_clocks(input int unsigned count);
    @(posedge clk_i);

    repeat (count-1) @(posedge clk_i);
  endtask

  //****************************************************************
  event go;
  function void start_monitoring();
    -> go;
  endfunction

  // ****************************************************************************
  initial begin
    @go;
    forever begin
      @(posedge clk_i);
      monitored_trans = new("monitored_trans");
      do_monitor( );



      proxy.notify_transaction( monitored_trans );

    end
  end

  //****************************************************************
  // The configure() function is used to pass agent configuration
  // variables to the monitor BFM.  It is called by the monitor within
  // the agent at the beginning of the simulation.  It may be called
  // during the simulation if agent configuration variables are updated
  // and the monitor BFM needs to be aware of the new configuration
  // variables.
  //
    function void configure(sa_out_configuration

                            sa_out_configuration_arg
                            );
    initiator_responder = sa_out_configuration_arg.initiator_responder;
  // pragma uvmf custom configure begin
  // pragma uvmf custom configure end
  endfunction



  // ****************************************************************************
  task do_monitor();
    //
    // Available struct members:
    //     //    monitored_trans.result
    //     //
    // Reference code;
    //    How to wait for signal value
    //      while (control_signal === 1'b1) @(posedge clk_i);
    //
    //    How to assign a transaction variable, named xyz, from a signal.
    //    All available input signals listed.
    //      monitored_trans.xyz = en_o_i;  //
    //      monitored_trans.xyz = result_o_i;  //
    // pragma uvmf custom do_monitor begin
    monitored_trans.start_time = $time;
    do @(negedge clk_i); while (en_o_i !== 1'b1);
    monitored_trans.result[2] = result_o_i;
    @(negedge clk_i);
    monitored_trans.result[1] = result_o_i;
    @(negedge clk_i);
    monitored_trans.result[0] = result_o_i;
    monitored_trans.end_time = $time;

    // pragma uvmf custom do_monitor end
```

```
  endtask


endinterface

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

sa_in_random_sequence.svh

Source path: UVMF/verification_ip/interface_packages/sa_in_pkg/src/sa_in_random_sequence.svh

```
//------------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//------------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//------------------------------------------------------------------------
//------------------------------------------------------------------------
//
// DESCRIPTION:
// This sequences randomizes the sa_in transaction and sends it
// to the UVM driver.
//
// This sequence constructs and randomizes a sa_in_transaction.
//
//------------------------------------------------------------------------
//------------------------------------------------------------------------
//
class sa_in_random_sequence
  extends sa_in_sequence_base ;

  `uvm_object_utils( sa_in_random_sequence )

  // pragma uvmf custom class_item_additional begin
  // pragma uvmf custom class_item_additional end

  //****************************************************************
  function new(string name = "");
    super.new(name);
  endfunction: new

  // ****************************************************************************
  // TASK : body()
  // This task is automatically executed when this sequence is started using the
  // start(sequencerHandle) task.
  //
  task body();
    // Exhaustively test 16 operand combinations (0..3 x 0..3)
    // followed by 15 randomized operand pairs.
    for (int a=0; a<4; a++) begin
      for (int b=0; b<4; b++) begin
        req = sa_in_transaction::type_id::create("exhaustive_req");
        start_item(req);
        req.ina = 2'(a);
        req.inb = 2'(b);
        finish_item(req);
      end
    end
    repeat (15) begin
      req = sa_in_transaction::type_id::create("random_req");
      start_item(req);
      if (!req.randomize()) `uvm_fatal("SA_RAND", "Cannot randomize")
      finish_item(req);
    end
  endtask

endclass: sa_in_random_sequence

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

sa_sa_predictor.svh

Source path: UVMF/verification_ip/environment_packages/sa_env_pkg/src/sa_sa_predictor.svh

```
// Serial-adder reference model; called from monitor TLM analysis path.
`uvm_analysis_imp_decl(_sa_input)
class sa_sa_predictor extends uvm_component;
  `uvm_component_utils(sa_sa_predictor)
  uvm_analysis_imp_sa_input #(sa_in_transaction, sa_sa_predictor) input_export;
  uvm_analysis_port #(sa_out_transaction) expected_ap;
  function new(string name, uvm_component parent);
```

```
    super.new(name,parent);
    input_export = new("input_export",this);
    expected_ap = new("expected_ap",this);
  endfunction
  function void write_sa_input(sa_in_transaction t);
    sa_out_transaction e;
    e = sa_out_transaction::type_id::create("expected");
    e.result = {1'b0,t.ina} + {1'b0,t.inb};
    expected_ap.write(e);
  endfunction
endclass
```

sa_sa_scoreboard.svh

Source path: UVMF/verification_ip/environment_packages/sa_env_pkg/src/sa_sa_scoreboard.svh

```
class sa_sa_scoreboard extends uvm_component;
 `uvm_component_utils(sa_sa_scoreboard)
 uvm_tlm_analysis_fifo #(sa_out_transaction) expected_fifo;
 uvm_tlm_analysis_fifo #(sa_out_transaction) actual_fifo;
 int unsigned pass_count, fail_count;
 function new(string name,uvm_component parent); super.new(name,parent); endfunction
 function void build_phase(uvm_phase phase);
   super.build_phase(phase);
   expected_fifo = new("expected_fifo",this);
   actual_fifo = new("actual_fifo",this);
 endfunction
 task run_phase(uvm_phase phase);
   sa_out_transaction exp, act;
   forever begin
     expected_fifo.get(exp);
     actual_fifo.get(act);
     if (exp.result === act.result) begin
       pass_count++;
       `uvm_info("SA_MATCH", $sformatf("sum=%0d",act.result), UVM_LOW)
     end else begin
       fail_count++;
       `uvm_error("SA_MISMATCH", $sformatf("expected=%0d actual=%0d",exp.result,act.result))
     end
   end
 endtask
 function void check_phase(uvm_phase phase);
   super.check_phase(phase);
   if (pass_count+fail_count == 0) `uvm_error("SA_NO_TEST","No transactions compared")
   if (expected_fifo.used() || actual_fifo.used()) `uvm_error("SA_REMAINDER","Unmatched transactions")
 endfunction
 function void report_phase(uvm_phase phase);
   `uvm_info("SA_RESULTS",$sformatf("pass=%0d fail=%0d",pass_count,fail_count),UVM_NONE)
 endfunction
endclass
```

sa_sa_coverage.svh

Source path: UVMF/verification_ip/environment_packages/sa_env_pkg/src/sa_sa_coverage.svh

```
class sa_sa_coverage extends uvm_subscriber #(sa_in_transaction);
  `uvm_component_utils(sa_sa_coverage)
  bit [1:0] last_a, last_b;
  covergroup operands_cg;
    a_cp: coverpoint last_a { bins vals[] = {[0:3]}; }
    b_cp: coverpoint last_b { bins vals[] = {[0:3]}; }
    all_pairs: cross a_cp, b_cp;
  endgroup
  function new(string name, uvm_component parent);
    super.new(name,parent);
    operands_cg = new();
  endfunction
  function void write(sa_in_transaction t);
    last_a=t.ina;
    last_b=t.inb;
    operands_cg.sample();
  endfunction
endclass
```

sa_environment.svh

Source path: UVMF/verification_ip/environment_packages/sa_env_pkg/src/sa_environment.svh

```
//------------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//------------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//------------------------------------------------------------------------
//------------------------------------------------------------------------
//
// DESCRIPTION: This environment contains all agents, predictors and
// scoreboards required for the block level design.
//
```

```
//------------------------------------------------------------------------
//------------------------------------------------------------------------
//

class sa_environment  extends uvmf_environment_base #(
    .CONFIG_T( sa_env_configuration
  ));
  `uvm_component_utils( sa_environment )









  typedef sa_in_agent  sa_in_agent_t;
  sa_in_agent_t sa_in_agent;

  typedef sa_out_agent  sa_out_agent_t;
  sa_out_agent_t sa_out_agent;









  typedef uvmf_virtual_sequencer_base #(.CONFIG_T(sa_env_configuration)) sa_vsqr_t;
  sa_vsqr_t vsqr;

  // pragma uvmf custom class_item_additional begin
  sa_sa_predictor sa_predictor;
  sa_sa_scoreboard sa_scoreboard;
  sa_sa_coverage sa_coverage;
  // pragma uvmf custom class_item_additional end

// ****************************************************************************
// FUNCTION : new()
// This function is the standard SystemVerilog constructor.
//
  function new( string name = "", uvm_component parent = null );
    super.new( name, parent );
  endfunction

// ****************************************************************************
// FUNCTION: build_phase()
// This function builds all components within this environment.
//
  virtual function void build_phase(uvm_phase phase);
// pragma uvmf custom build_phase_pre_super begin
// pragma uvmf custom build_phase_pre_super end
    super.build_phase(phase);
    sa_in_agent = sa_in_agent_t::type_id::create("sa_in_agent",this);
    sa_in_agent.set_config(configuration.sa_in_agent_config);
    sa_out_agent = sa_out_agent_t::type_id::create("sa_out_agent",this);
    sa_out_agent.set_config(configuration.sa_out_agent_config);

    vsqr = sa_vsqr_t::type_id::create("vsqr", this);
    vsqr.set_config(configuration);
    configuration.set_vsqr(vsqr);

    // pragma uvmf custom build_phase begin
    sa_predictor = sa_sa_predictor::type_id::create("sa_predictor",this);
    sa_scoreboard = sa_sa_scoreboard::type_id::create("sa_scoreboard",this);
    sa_coverage = sa_sa_coverage::type_id::create("sa_coverage",this);
    // pragma uvmf custom build_phase end
  endfunction

// ****************************************************************************
// FUNCTION: connect_phase()
// This function makes all connections within this environment.  Connections
// typically inclue agent to predictor, predictor to scoreboard and scoreboard
// to agent.
//
  virtual function void connect_phase(uvm_phase phase);
// pragma uvmf custom connect_phase_pre_super begin
// pragma uvmf custom connect_phase_pre_super end
    super.connect_phase(phase);
    // Custom UVMF TLM integration: monitor -> predictor -> expected FIFO;
    //                              output monitor -> actual FIFO.
    sa_in_agent.monitored_ap.connect(sa_predictor.input_export);
    sa_in_agent.monitored_ap.connect(sa_coverage.analysis_export);
```

```
    sa_predictor.expected_ap.connect(sa_scoreboard.expected_fifo.analysis_export);
    sa_out_agent.monitored_ap.connect(sa_scoreboard.actual_fifo.analysis_export);
    // pragma uvmf custom reg_model_connect_phase begin
    // pragma uvmf custom reg_model_connect_phase end
  endfunction

// ****************************************************************************
// FUNCTION: end_of_simulation_phase()
// This function is executed just prior to executing run_phase.  This function
// was added to the environment to sample environment configuration settings
// just before the simulation exits time 0.  The configuration structure is
// randomized in the build phase before the environment structure is constructed.
// Configuration variables can be customized after randomization in the build_phase
// of the extended test.
// If a sequence modifies values in the configuration structure then the sequence is
// responsible for sampling the covergroup in the configuration if required.
//
  virtual function void start_of_simulation_phase(uvm_phase phase);
     configuration.sa_configuration_cg.sample();
  endfunction

endclass

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

sa_env_configuration.svh

Source path: UVMF/verification_ip/environment_packages/sa_env_pkg/src/sa_env_configuration.svh

```
//------------------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//------------------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//------------------------------------------------------------------------------
//------------------------------------------------------------------------------
//
// DESCRIPTION: THis is the configuration for the sa environment.
//  it contains configuration classes for each agent.  It also contains
//  environment level configuration variables.
//
//------------------------------------------------------------------------------
//------------------------------------------------------------------------------
//
class sa_env_configuration
extends uvmf_environment_configuration_base;

  `uvm_object_utils( sa_env_configuration )


//Constraints for the configuration variables:



  covergroup sa_configuration_cg;
    // pragma uvmf custom covergroup begin
    option.auto_bin_max=1024;
    // pragma uvmf custom covergroup end
  endgroup



    typedef sa_in_configuration sa_in_agent_config_t;
    rand sa_in_agent_config_t sa_in_agent_config;

    typedef sa_out_configuration sa_out_agent_config_t;
    rand sa_out_agent_config_t sa_out_agent_config;






  typedef uvmf_virtual_sequencer_base #(.CONFIG_T(sa_env_configuration)) sa_vsqr_t;
  sa_vsqr_t vsqr;

  // pragma uvmf custom class_item_additional begin
  // pragma uvmf custom class_item_additional end

// ****************************************************************************
// FUNCTION : new()
// This function is the standard SystemVerilog constructor.
// This function constructs the configuration object for each agent in the environment.
//
```

```
  function new( string name = "" );
    super.new( name );



    sa_in_agent_config = sa_in_agent_config_t::type_id::create("sa_in_agent_config");
    sa_out_agent_config = sa_out_agent_config_t::type_id::create("sa_out_agent_config");



    sa_configuration_cg=new;
    `uvm_warning("COVERAGE_MODEL_REVIEW", "A covergroup has been constructed which may need review because of either generation or re-
generation with merging.  Please note that configuration variables added as a result of re-generation and merging are not automatically added
to the covergroup.  Remove this warning after the covergroup has been reviewed.")


  // pragma uvmf custom new begin
  // pragma uvmf custom new end
  endfunction

// ****************************************************************************
// FUNCTION : set_vsqr()
// This function is used to assign the vsqr handle.
  virtual function void set_vsqr( sa_vsqr_t vsqr);
     this.vsqr = vsqr;
  endfunction : set_vsqr

// ****************************************************************************
// FUNCTION: post_randomize()
// This function is automatically called after the randomize() function
// is executed.
//
  function void post_randomize();
    super.post_randomize();
    // pragma uvmf custom post_randomize begin
    // pragma uvmf custom post_randomize end
  endfunction

// ****************************************************************************
// FUNCTION: convert2string()
// This function converts all variables in this class to a single string for
// logfile reporting. This function concatenates the convert2string result for
// each agent configuration in this configuration class.
//
  virtual function string convert2string();
    // pragma uvmf custom convert2string begin
    return {

     "\n", sa_in_agent_config.convert2string,
     "\n", sa_out_agent_config.convert2string



       };
    // pragma uvmf custom convert2string end
  endfunction
// ****************************************************************************
// FUNCTION: initialize();
// This function configures each interface agents configuration class.  The
// sim level determines the active/passive state of the agent.  The environment_path
// identifies the hierarchy down to and including the instantiation name of the
// environment for this configuration class.  Each instance of the environment
// has its own configuration class.  The string interface names are used by
// the agent configurations to identify the virtual interface handle to pull from
// the uvm_config_db.
//
  function void initialize(uvmf_sim_level_t sim_level,
                                    string environment_path,
                                    string interface_names[],
                                    uvm_reg_block register_model = null,
                                    uvmf_active_passive_t interface_activity[] = {}
                                   );

    super.initialize(sim_level, environment_path, interface_names, register_model, interface_activity);



  // Interface initialization for local agents
     sa_in_agent_config.initialize( interface_activity[0], {environment_path,".sa_in_agent"}, interface_names[0]);
     sa_in_agent_config.initiator_responder = INITIATOR;
     // sa_in_agent_config.has_coverage = 1;
     sa_out_agent_config.initialize( interface_activity[1], {environment_path,".sa_out_agent"}, interface_names[1]);
     sa_out_agent_config.initiator_responder = RESPONDER;
     // sa_out_agent_config.has_coverage = 1;
```

```
  // pragma uvmf custom initialize begin
  // pragma uvmf custom initialize end

  endfunction

endclass

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

sa_env_pkg.sv

Source path: UVMF/verification_ip/environment_packages/sa_env_pkg/sa_env_pkg.sv

```
//----------------------------------------------------------------------
// Created with uvmf_gen version 2023.4
//----------------------------------------------------------------------
// pragma uvmf custom header begin
// pragma uvmf custom header end
//----------------------------------------------------------------------
//----------------------------------------------------------------------
//
// PACKAGE: This file defines all of the files contained in the
//    environment package that will run on the host simulator.
//
// CONTAINS:
//     - <sa_configuration.svh>
//     - <sa_environment.svh>
//     - <sa_env_sequence_base.svh>
//
//----------------------------------------------------------------------
//----------------------------------------------------------------------
//
package sa_env_pkg;

  import uvm_pkg::*;
  `include "uvm_macros.svh"
  import uvmf_base_pkg::*;
  import sa_in_pkg::*;
  import sa_in_pkg_hdl::*;
  import sa_out_pkg::*;
  import sa_out_pkg_hdl::*;



  // pragma uvmf custom package_imports_additional begin
  // pragma uvmf custom package_imports_additional end

  // Parameters defined as HVL parameters

  `include "src/sa_env_typedefs.svh"
  `include "src/sa_env_configuration.svh"
  `include "src/sa_environment.svh"
  `include "src/sa_env_sequence_base.svh"

  // pragma uvmf custom package_item_additional begin
  `include "src/sa_sa_predictor.svh"
  `include "src/sa_sa_scoreboard.svh"
  `include "src/sa_sa_coverage.svh"
  // UVMF_CHANGE_ME : When adding new environment level sequences to the src directory
  //    be sure to add the sequence file here so that it will be
  //    compiled as part of the environment package.  Be sure to place
  //    the new sequence after any base sequence of the new sequence.
  // pragma uvmf custom package_item_additional end

endpackage

// pragma uvmf custom external begin
// pragma uvmf custom external end
```

README.md

Source path: UVMF/custom/README.md

```
# Customization requirements

These are **integration reference snippets**, not generator-consumed YAML.
Copy each into the corresponding UVMF-generated `// UVMF_CHANGE_ME` or user-code region **after inspecting generated method signatures**. The
generator cannot derive the serial protocol drive/monitor timing, predictor logic, or scoreboard connections from the four structural YAMLs.
```

```
1. `sa_in_driver_bfm_tasks.sv.txt`: serial stimulus on negedges.
2. `sa_out_monitor_bfm_tasks.sv.txt`: collect three output bits following `en_o`.
3. `sa_predictor.svh`: expected sum calculation. Reference uses pure SV; original C++ DPI predictor is in `original_uvm/` and can be retained
with `-sv_lib`.
4. `sa_scoreboard.svh`: expected/actual result comparisons.
5. `sa_sequence.svh`: 15 randomized transactions.
6. `sa_coverage.svh`: 16-bin operand cross.

Generated UVMF agent BFMs use version-specific proxy APIs. These snippets are intentionally not asserted to compile unchanged; map them to
the generator output. Generated environment requires explicit predictor/scoreboard creation and TLM connection.
```

CUSTOM_INTEGRATION_STATUS.md

Source path: UVMF/CUSTOM_INTEGRATION_STATUS.md

```
# SA UVMF 2023.4 — custom integration

## Changes applied directly to generated source

- `sa_in_driver_bfm.sv`: drives first and second input bits with enable strobed at the first bit. Delays further transactions until serial
calculation is complete.
- `sa_in_monitor_bfm.sv`: reconstructs 2-bit A and B on falling clock edges.
- `sa_out_monitor_bfm.sv`: reconstructs 3-bit sum beginning on `en_o`, matching the original UVM output-monitor convention.
- `sa_sa_predictor.svh`: maps input transactions to expected 3-bit sum.
- `sa_sa_scoreboard.svh`: consumes expected and actual FIFOs; reports comparison pass/fail.
- `sa_sa_coverage.svh`: functional operand-pair coverage (16 bins).
- `sa_environment.svh`: instantiates predictor, scoreboard, coverage; makes monitor-to-predictor/FIFO/coverage TLM connections.
- `sa_env_pkg.sv`: includes additional components.
- `sa_in_random_sequence.svh`: exercises all 16 input combinations plus 15 random transactions.

## Critical RTL compatibility note

The provided `sa.vhd` has no reset input. Generated BFM reset (`rst_n`) is solely testbench-side. Generated bench and DUT binding/top-level
remain **outside this archive**, and must be generated or supplied separately; build/test execution is not validated.

## Outstanding integration

1. Regenerate bench from `yaml/sa_bench.yaml` or restore your existing UVMF bench/test infrastructure. Connect sa_in_if/sa_out_if to VHDL
DUT; do not connect rst_n to the DUT.
2. Start `sa_in_random_sequence` on the input agent's sequencer from your UVMF test/virtual sequence. Allow output transactions to drain
before ending objection.
3. Confirm UVMF 2023.4 generated base library exposes `monitored_ap` and that the environment compiles.
4. Run coverage, scoreboard comparisons, and Questa compile/simulation; adjust serial output sampling only if a waveform establishes mismatch
with the original protocol.
5. Regeneration of `verification_ip` may overwrite modifications; reapply custom portions or move into generator-preserved regions.

**Verification status:** static integration only, not compile-verified, not simulation-verified.
```

Appendix B. File map and reading order

The source archive contains generated HDL and HVL packages in verification_ip/interface_packages, the generated environment in verification_ip/environment_packages, generator inputs in yaml, the DUT in rtl, custom alternative snippets in custom, and the previous UVM implementation in original_uvm. Read YAML → transaction → HDL BFM → HVL monitor → predictor → scoreboard → environment → bench/top → test.

Original UVM concept	UVMF counterpart
sa_transaction.svh	sa_in_transaction.svh / sa_out_transaction.svh
sa_driver.sv	sa_in_driver.svh and sa_in_driver_bfm.sv
sa_monitor.sv	sa_in_monitor_bfm.sv / sa_out_monitor_bfm.sv and HVL monitors
sa_scoreboard.sv	sa_sa_scoreboard.svh
sa_env.sv	sa_environment.svh
sa_test.sv	Test startup remains to be integrated
sa_tb_top.sv	Bench/DUT binding remains to be integrated